

Unity Manual

Instantiate -> Used for Cloning the Object

Syntax - Instantiate(gameobject_name)

To declare gameobject -

private GameObject <variable>

Eg - private GameObject _playerPrefab

To declare the Gameobject that could store list of items -

Syntax - private GameObject[] <var_name>

E. g- private GameObject[] _powerups

To declare a script -

private <name_of_script> <variable_name>

To declare the component to access -

private <name_of_component> <variable_name>

Eg - Private SpawnManager _spawnManager

Quaternion.identity - Used to handle rotation of the gameobject

Vector3 -> Used to define position of the gameobject

Syntax - new Vector3(x_cordinate , y_cordinate , z_cordinate)

E. g - new Vector3 (2, 3.5f, 0)

Translate -> Used to move the gameobject

Syntax - transform.Translate(Position , _speed (extra-parmeters))

E/ g - transform.Translate(Vector3.down * speed)

UI_Manager – Used to handle to UI of the game

To Move the gameobject on key press –

Syntax – `Keyboard.current.(key)Key.isPressed`

E. g – `Keyboard.current.wKey.isPressed`

Panel –

It is a gameobject used to group certain Ui elements into one group. For e.g. – Resume Panel, Exit Panel. It is always recommended to be children of Canvas gameobject

`transform.position` – Used to get the position of gameobject

`transform.position.x(value)` – Used (<) when the gameobject goes vertically down below zero

If want to position the gameobject to the new variable –

Syntax – `transform.position + new Vector3( x , y , z)`

E.g – `transform.position + new Vector3(0 , 2 , 0)`

`Destroy(this.gameobject)` – Used to remove the gameobject attached to the script from the scene

`Time.time` – Starts the game clock as soon as the game starts from 0 sec

`Random.Range` – It is used to randomize the positions between specified coordinates

Syntax – `Random.Range(from, to)`

E. g – `Random.Range(8f, -8f)`

Other.transform.name is publicly accessible

*To define the component to use, we write -
Syntax - private <component_name> <variable>
E. g - private SpawnManager _spawnManager*

To access the method of the component -

Syntax -

`other.transform.GetComponent<component_name>().<method_name>()`

E. g - `other.transform.GetComponent<Player>.Damage()`

OR

Syntax - `<variable_name> =`

`GameObject.Find("<gameobject_name>").GetComponent"<Component_name>";`

E.g - `_player = GameObject.Find("Player").GetComponent<Player>();`

Co-routines -

It is used to delay events and execute a piece of code after some time

Allows to create a sequence of events

IEnumerator Keyword is used

To start a co-routine -

`StartCoroutine(<Coroutine_Name>)`

OR

`StartCoroutine("<Coroutine_name>")`

`Yield return WaitForSeconds(5f) ->` Used to start the coroutine again after the specified time

Sorting Layer -

It is used to bring the objects to the foreground or to the background, By Default, the sorting is set to background. You can toggle the sorting layer through the inspector tab , under sprites section

0 - Background, 1 - Foreground

Animation -

It is used to apply effects(animation) to the gameobject

Select gameobject -> Windows -> Animation - > Animation -> Create -> Name the animation

Drag the animation (sequence of gameobject) to the dopesheet

Hit the record button and play the animation

Gameobject_name.controller file is the main file that controls the logic of the animation

To stop playing the animation as soon as the gameobject gets spawned -

Create an empty State. Make it as a default state. Now make a transition from empty state to animation

To stop the animation from looping -

Select the animation of the gameobject (name by which animation saved) uncheck the loop time checkbox

To start the animation instantly after collision -

Click on the arrow connecting empty state and the animation, uncheck the "Has Exit Time" checkbox

Triggers -

To begin some animation on some events like. These events are called triggers. Triggers can be set through parameters, to define a parameter -

Click on Animation -> Go to Parameters Section -> Click on (+) icon to add a trigger. Name the trigger and use it start some event.

To add the trigger to the animation -

Open the animation -> Click on the arrow connecting default state and the animation -> Click on (+) icon under conditions tab.

To enable the trigger -

```
_anim.SetTrigger("<trigger_name>")
```

Audio -

To play audio clip , apply component audio source to gameobject
Drag the audio to the audio clip

To play audio -

Syntax - <variable _name>.Play()

E. g - _audiSrc.Play()

To Play Clip at Specific Point -

Syntax - <Component _name>.PlayClipAtPoint(<variable name that has audio clip>, position)

To Assign the sound to the group of gameobject -

Select all the gameobject

Attach the audio source component to gameobject

To Destroy gameobject after spawning, we write -

```
if(transform.parent != null) {  
    Destroy(transform.parent.gameObject)  
}
```

Post Processing -

Apply effects to improve appearance of the game

Create an empty gameobject -> apply the Post Processing Volume

To apply the on the whole game, check option "Is Global".

Create a new Profile in the Post Processing Volume

Download the post process volume package from package manager by navigating -

Window -> Package Management -> Package Manager

Add the post-process Layer component to the main camera

Apply the new Profile to layer parameter in the main camera

To Store the data -

We use PlayerPrefs class which is a data storage system used to store preferences (data like highscore etc). It can store only primitive data types like int, float, bool.

Syntax -> `PlayerPrefs.GetInt(<key_name> , <default_value>)`

Syntax -> `PlayerPrefs.SetInt(<key_name> , <value_to_store>)`

Eg `_bestscore = PlayerPrefs.GetInt("BestScore" , 0)`

It retrieves the value with the key(BestScore)

Eg - `PlayerPrefs.SetInt("BestScore" , _BestScore);`